Hybrid RAG Knowledge Assistant

Major Project Report

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY IN

COMPUTER SCIENCE AND ENGINEERING

SUBMITTED BY:

ABHAY MAURYA (221620101001) **AMIT YADAV** (221620101010)

VARUN RANA (221620101059) **SHANU AHMED** (731620101003)

SAMIR AHMAD (731620101005)

Under the able guidance of

Assistant Prof**. Mrs. Sneh Lata Singh**

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DR. A.P.J. ABDUL KALAM INSTITUTE OF TECHNOLOGY

TANAKPUR (CHAMPAWAT, UTTARAKHAND) – 262309

(CAMPUS INSTITUTE OF)

VEER MADHO SINGH BHANDARI UTTARAKHAND TECHNICAL UNIVERSITY, DEHRADUN

February – July, 2026

CERTIFICATE

This is to certify that the Major Project entitled **"Hybrid RAG Knowledge Assistant"** has been successfully completed and submitted by the following students:

Group Members	**Roll No.**
Abhay Maurya	221620101001
Amit Yadav	221620101010
Varun Rana	221620101059
Shanu Ahmed	731620101003
Samir Ahmad	731620101005

This Major Project is a record of the students' original work carried out by them in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering from Dr. A.P.J. Abdul Kalam Institute of Technology, Tanakpur (Campus Institute of VMSBUTU, Dehradun), Uttarakhand, India.

Mrs. Sneh Lata Singh
(Project Mentor)

Mr. Naresh Kumar
(Head of Department, CSE)

(External Examiner)

ACKNOWLEDGEMENT

We would like to express our deepest appreciation to our institute's director, **Prof. Hardwari Lal Mandoria**, whose knowledge and astute criticism have been crucial in forming this project and guaranteeing its successful conclusion.

We would like to express our deepest gratitude to our project mentor, **Mrs. Sneh Lata Singh**, for his valuable guidance, constant encouragement, and insightful suggestions throughout the development of our Major project **"Hybrid RAG Knowledge Assistant."** His expertise, patience, and constructive feedback played a crucial role in shaping this project and bringing it to successful completion.

We extend our sincere thanks to **Mr. Naresh Kumar**, Head of the Department (CSE), for his continuous support, motivation, and for providing us with the facilities and environment necessary for the successful execution of this project.

Finally, we gratefully acknowledge the contributions of **Mr. Rishabh Kushawah, Mr. Divisth Jaiswal, Ms. Neha Bisht, Mrs. Komal Gahtori, and Mrs. Shreya Bisht** for their academic guidance and constant support during the entire course of this Major project.

We would also like to convey our heartfelt gratitude to our classmates, instructors, and friends for their cooperation, valuable inputs, and encouragement throughout this project. Their moral support and feedback have greatly contributed to the refinement of our work.

Name	Roll No.	Signature
Abhay Maurya	221620101001	.....
Amit Yadav	221620101010	.....
Varun Rana	221620101059	.....
Shanu Ahmed	731620101003	.....
Samir Ahmad	731620101005	.....

LIST OF FIGURES

ABBREVIATIONS

API	**Application Programming Interface (Connects Frontend to Backend)**
CSS	**Cascading Style Sheets (Used for UI styling)**
DB	**Database (Stores users and chat logs)**
ERD	**Entity-Relationship Diagram (Visualizes data structure)**
HTML	**HyperText Markup Language (Used for web structure)**
JS	**JavaScript (Used for frontend logic and interactivity)**
JSON	**JavaScript Object Notation (Format for data exchange)**
LLM	**Large Language Model (The AI "brains" like Llama 3 or Gemini)**
NLP	**Natural Language Processing (The technology behind understanding text)**
PDF	**Portable Document Format (The primary document input type)**
RAG	**Retrieval-Augmented Generation (The core architecture of your project)**
SDLC	**Software Development Life Cycle (The Agile process used)**
UI / UX	**User Interface / User Experience (The web design and feel)**
WBS	**Work Breakdown Structure (The task hierarchy of your project)**

ABSTRACT

The **Hybrid Retrieval-Augmented Generation (RAG) Knowledge Assistant** is a privacy-focused system designed to enable users to securely upload, manage, and query their own documents using artificial intelligence. Unlike traditional AI tools that rely on cloud-based processing and pose risks to sensitive data, this system introduces a hybrid architecture that combines both offline and online AI models. For confidential information, a local Large Language Model operates in offline mode, ensuring that data remains within the user's device or private server, while non-sensitive queries can leverage powerful cloud-based models for improved performance and reasoning. The system uses the RAG approach to retrieve relevant information from user-uploaded documents and generate accurate, context-aware responses, reducing hallucinations and improving reliability. Additionally, the system incorporates intelligent document indexing, semantic search, and efficient data retrieval techniques to enhance response speed and accuracy. It is designed with a user-friendly interface that allows easy document upload, query handling, and seamless switching between offline and online modes. This solution is highly scalable and can be adapted for various domains such as education, healthcare, legal, and corporate environments where data privacy is critical. Overall, it provides a secure, flexible, and efficient platform for interacting with personal and organizational data, making it a robust and user-controlled next-generation AI assistant.

TABLE OF CONTENT

Chapter 1

INTRODUCTION

The **Hybrid RAG Knowledge Assistant** is a privacy-first document question-answering system designed to overcome the data security risks and hallucination issues associated with standard cloud-based AI. By utilizing Retrieval-Augmented Generation (RAG), the system allows users to upload custom documents and generates highly accurate answers based strictly on that provided context rather than pre-trained guesswork. Its core innovation is a dual-engine architecture that provides complete control over data privacy: an offline "Private Mode" that runs local models (like Ollama) for confidential files, ensuring sensitive information never leaves the device, and an online "Smart Mode" that leverages powerful cloud AI for faster, advanced reasoning on non-sensitive queries. Ultimately, this project delivers a secure, context-aware AI assistant that seamlessly balances uncompromising data privacy with cutting-edge artificial intelligence.

1.1 Problem Statement

Current AI platforms require users to upload documents to external cloud servers, creating severe privacy risks for confidential and sensitive data. Additionally, standard AI models often hallucinate or provide inaccurate answers because they cannot directly reference a user's private files. There is a critical need for a flexible system that allows users to accurately query their own documents while maintaining complete control over data privacy through offline processing.

- **Data Privacy Risks:** Public AI tools require uploading sensitive documents to external cloud servers, compromising the security of confidential company or personal data.

- **AI Hallucinations:** Standard models often guess incorrect answers because they lack direct access to the user's specific files and context.
- **Lack of Control:** Users currently lack a flexible system that allows them to process highly sensitive data entirely offline while still having the option to use powerful cloud AI for non-sensitive queries.

1.2 Objectives

The main objectives of the proposed project are:

- To develop a **Hybrid RAG Knowledge Assistant** for interacting with user-uploaded documents.
- To ensure **data privacy and security** by enabling offline processing using local AI models.
- To implement **Retrieval-Augmented Generation (RAG)** for accurate and context-based responses.
- To integrate both **offline and online AI systems** with intelligent switching based on data sensitivity.
- To design a **user-friendly and efficient interface** for seamless document upload, search, and query handling.

Chapter 2

LITERATURE REVIEW

The evolution of Large Language Models (LLMs) has revolutionized information retrieval, yet enterprise adoption is hindered by severe data privacy risks—as public models process sensitive information on external cloud servers—and recurring "hallucination" issues where models generate factually incorrect responses. To mitigate these inaccuracies, the Retrieval-Augmented Generation (RAG) framework was developed, allowing AI to ground its answers strictly in user-provided documents via vector search rather than relying solely on pre-trained memory. Concurrently, the rise of edge computing and highly efficient local AI models (such as Ollama) has enabled fully offline data processing to ensure zero data leakage. This project synthesizes these advancements by proposing a Hybrid AI architecture that dynamically routes tasks: it utilizes the factual accuracy of RAG alongside a dual-engine system that processes confidential data via secure offline models and non-sensitive data via powerful cloud APIs, thereby achieving an optimal balance between uncompromising privacy and advanced computational reasoning.

2.1 Summary of Existing Solutions

- **Public Cloud LLMs (e.g., ChatGPT, Gemini):** Offer highly advanced reasoning and fast processing speeds but require uploading data to external servers, making them entirely unsuitable for confidential or proprietary documents.
- **Standard RAG Implementations:** Improve factual accuracy by allowing users to chat with their documents, but the majority of existing solutions still rely on cloud-based APIs to generate the final response, failing to resolve the core privacy issue.

- **Fully Local AI Frameworks:** Utilize tools like Ollama or GPT4All to run open-source models completely offline, guaranteeing data privacy but often suffering from slower processing speeds and lower reasoning capabilities compared to their cloud-based counterparts.

2.2 Identification of Research Gaps

- **Lack of Dynamic Sensitivity Routing:** Existing solutions force users into a binary choice: either process everything in the cloud (risking privacy) or process everything locally (sacrificing speed and advanced reasoning). There is a gap in systems that intelligently toggle between local and cloud processing based on data sensitivity.
- **Absence of a Unified Hybrid Environment:** There is a lack of cohesive, user-friendly platforms that seamlessly integrate offline vector databases, local LLMs, and cloud APIs into a single document-querying interface.
- **Resource Optimization in RAG:** Current localized RAG systems do not optimize computational resources by offloading non-sensitive queries to the cloud, resulting in unnecessary strain on local hardware.

Chapter 3

TECHNOLOGY AND METHODOLOGY

3.1 System Architecture

A web interface connects to a secure RAG pipeline featuring a hybrid router that processes sensitive queries locally and general queries via cloud AI.

3.1.1 Technology Stack

- **Frontend:** React/Bootstrap (HTML/CSS/JS)
- **Backend & Vector Store:** Python, LangChain, ChromaDB, FAISS
- **Hybrid AI:** Ollama (Local) & Gemini/OpenAI (Cloud)

3.1.2 AI & ML Components

- **Embeddings:** Local HuggingFace models for secure, offline vector generation.
- **Offline AI:** Ollama (Llama 3/Mistral) for private, on-device text generation.
- **Cloud AI:** Gemini or OpenAI APIs for advanced reasoning on non-sensitive **tasks.**
- **RAG Pipeline:** LangChain framework using cosine similarity for accurate semantic document retrieval.

3.1.3 Notification System

- **Security Alerts:** Immediate notifications when the system toggles between Offline (Private) and Online (Smart) modes.

3.1.4 Blog Page Implementation

- An integrated resource center to help users maximize the system's capabilities.
- Features articles on "Prompt Engineering," "RAG Basics," and "Data Privacy".

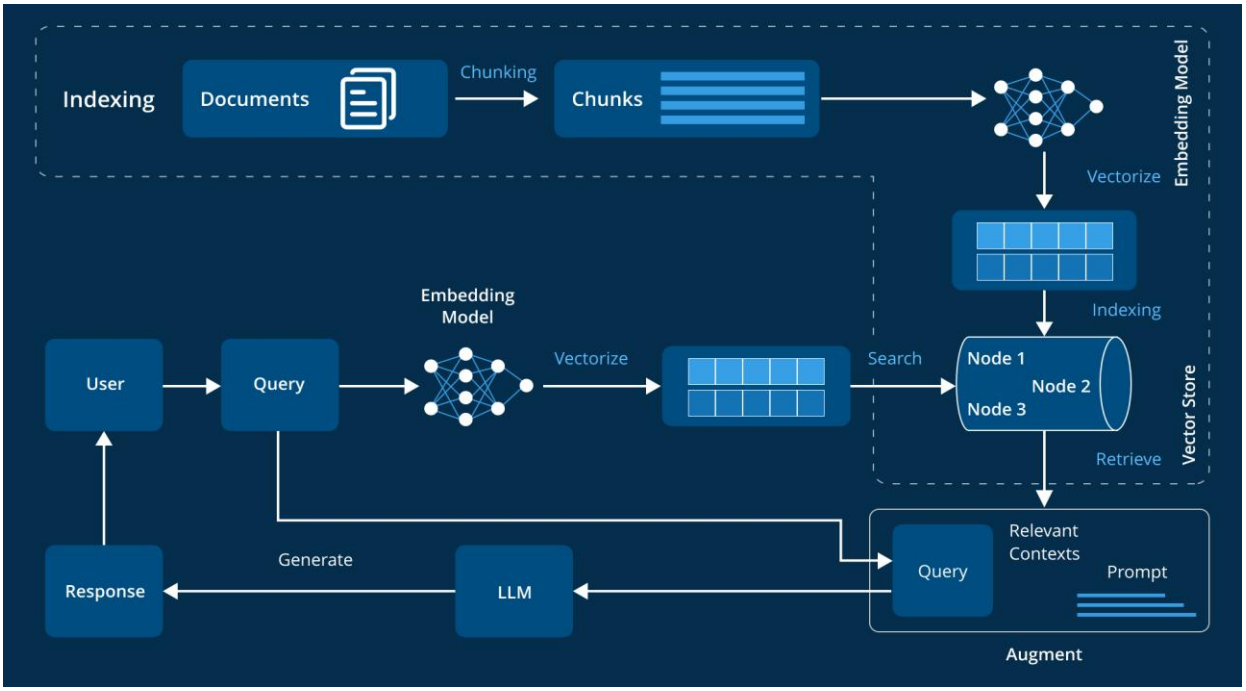

Figure 3.1.1: System Architecture Diagram for Hybrid RAG Knowledge Assistant .

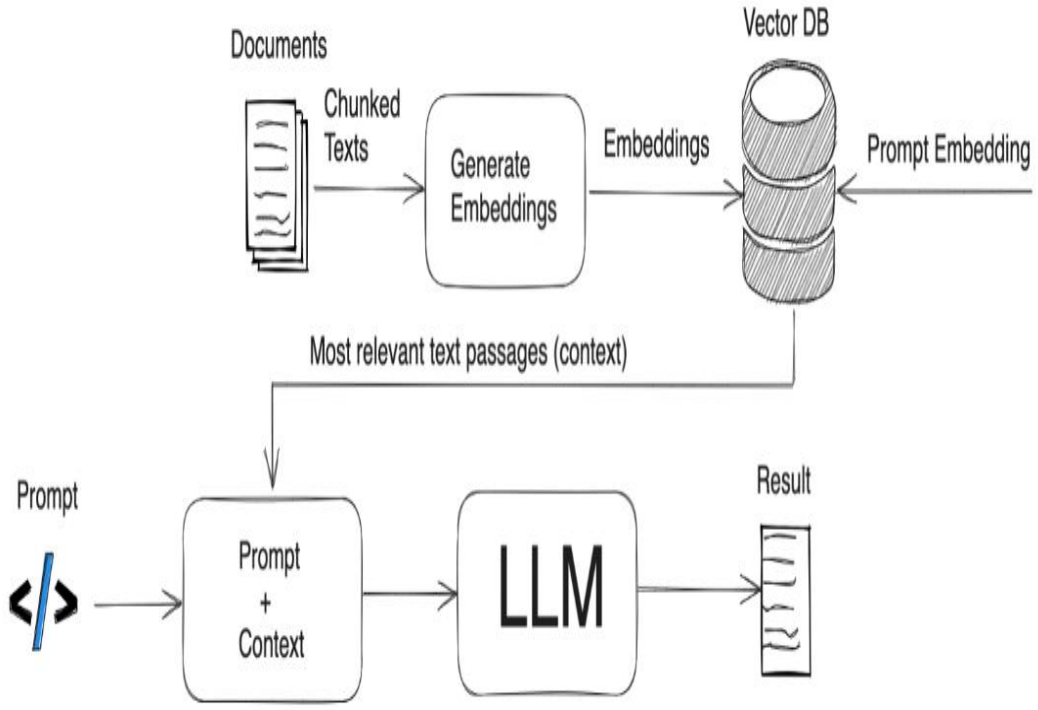

Figure 3.1.2: Workflow Diagram for Hybrid RAG Knowledge Assistant .

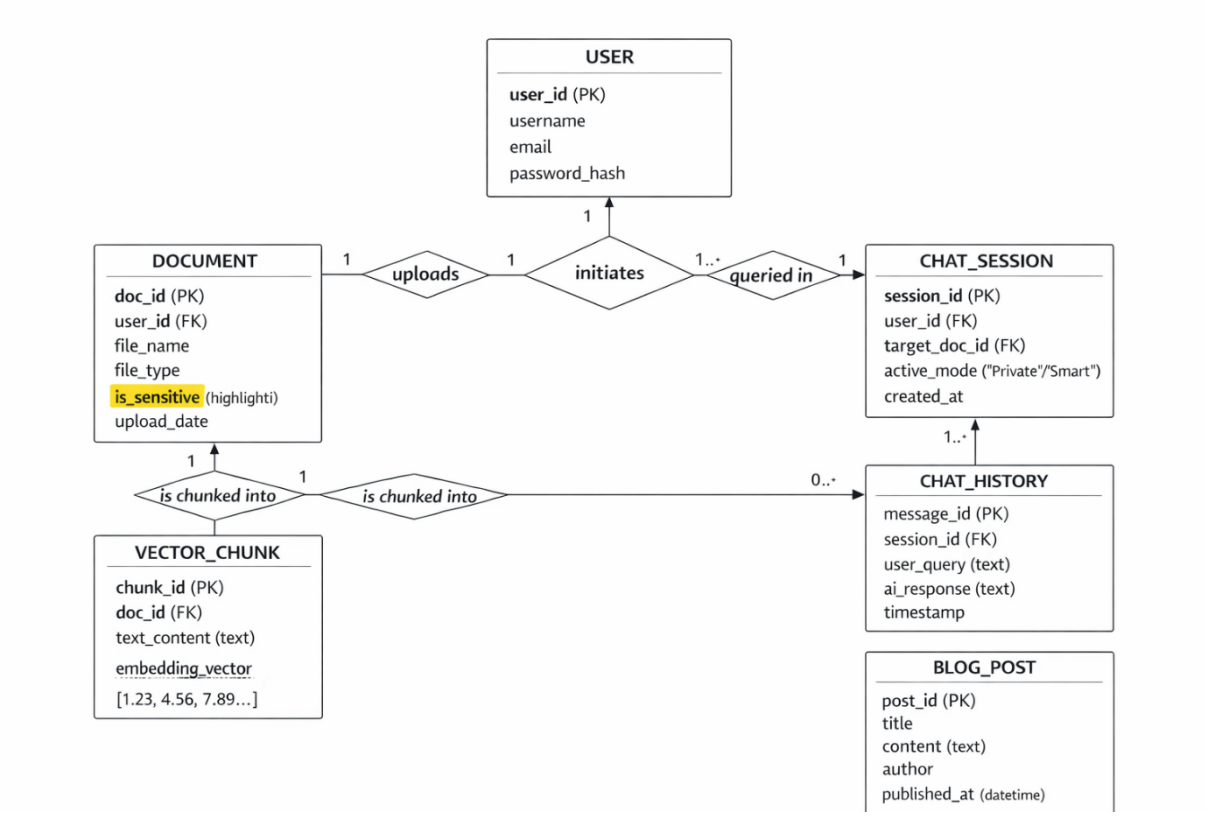

Figure 3.1.3: ER Diagram for Hybrid RAG Knowledge Assistant .

3.2Methodology

The **Hybrid RAG Knowledge Assistant** is an innovative, privacy-first AI system designed to resolve the data security risks and hallucination issues prevalent in standard Large Language Models. By utilizing a Retrieval-Augmented Generation (RAG) framework, the application allows users to upload customized documents and generates accurate answers based strictly on that private data. Built using Python, LangChain, and a ChromaDB vector database, the secure backend connects to an intuitive frontend web interface. The core innovation of the project is its dynamic hybrid routing architecture, which intelligently categorizes queries based on data sensitivity. For highly confidential files, the system operates entirely offline using local models via Ollama, guaranteeing zero internet data leakage. Conversely, for general queries, it seamlessly toggles to a cloud-based "Smart Mode" utilizing powerful APIs for advanced reasoning. Ultimately, this dual-engine approach

delivers a highly secure, context-aware AI assistant that ensures an optimal balance between uncompromising enterprise data privacy and cutting-edge computational intelligence.

3.3Software Requirements

To build and execute the Hybrid RAG Knowledge Assistant, a modern development stack is required that supports both web development and machine learning orchestration.

- **Operating System:** Windows 10/11, macOS, or Linux (Ubuntu is highly preferred for seamless local AI execution).
- **Programming Languages:** Python 3.9+ (for backend RAG pipeline and system logic) and standard HTML, CSS, and JavaScript (for the frontend user interface).
- **AI & Machine Learning Frameworks:** Ollama must be installed locally to serve the offline language models. The backend relies heavily on LangChain (or LlamaIndex) for RAG orchestration and vector database management.
- **Database:** A local vector database such as ChromaDB or FAISS to securely store and retrieve document embeddings.
- **Libraries & Packages:** Key Python dependencies include PyPDF2 (for document extraction), sentence-transformers (for HuggingFace embeddings), and FastAPI or Flask (for backend routing).
- **Development Tools:** Visual Studio Code (VS Code) or PyCharm as the primary IDE, and Git/GitHub for version control and project management.

3.4 Hardware Requirements

Because the Hybrid RAG Knowledge Assistant features a "Private Mode" that processes sensitive documents entirely offline, the system requires capable local hardware to run the Large Language Models (LLMs) and vector database smoothly.

- **Processor (CPU):** Intel Core i5 / AMD Ryzen 5 (8th Gen or newer, multi-core architecture recommended for parallel data processing).
- **Memory (RAM):** Minimum 8 GB is required to run the operating system and backend server, though **16 GB is highly recommended** to ensure smooth execution of local AI models (such as Llama 3 or Mistral) without system lag.
- **Storage:** Minimum 20 GB of free space. A Solid State Drive (SSD) is strongly recommended over an HDD to ensure rapid vector retrieval from the local database (ChromaDB) and fast loading of AI model weights.
- **Graphics Processing Unit (GPU):** While the system can run on a CPU, a dedicated graphics card (e.g., NVIDIA GPU with 6GB+ VRAM) is highly recommended to drastically accelerate text generation speeds and vector embedding processes during offline mode.

3.5 Work Breakdown Structure (WBS)

Table 1: Work Breakdown Structure

S.No.	Phase	Task	Description
1	Planning	Problem study & requirement analysis	Identify core requirements for dual-engine AI and zero-leakage offline processing.
2	Design	System architecture, database, and UI design	Create ER diagrams, Hybrid Router logic flow, and wireframes for the chat UI.
3	Development (Frontend)	Responsive web UI	Build HTML/CSS/JS (or React) interface with chat window, upload zone, and mode toggle.
4	Development (Backend)	API and core routing setup	Implement FastAPI/Flask endpoints, relational database, and secure file handling.
5	Core RAG Development	Document parsing and vector storage	Implement PyPDF extraction, text chunking, and ChromaDB vector embeddings.
6	AI Integration	Hybrid AI engine connection	Connect local Ollama (Offline Mode) and Gemini/OpenAI APIs (Smart Mode) with dynamic routing.
7	Testing	Unit, retrieval, and privacy testing	Validate RAG accuracy (zero hallucinations) and confirm completely secure offline execution.

3.6 Gantt Chart

Table 2: Gantt Chart

Week	Tasks	Details / Deliverables
Week 1	Requirement Analysis & System Design	Finalize project scope, prepare system architecture and ER diagrams, and configure the local development environment (Ollama, ChromaDB).
Week 2	Backend & Frontend Development	Develop Python backend APIs (FastAPI/Flask), initialize the vector database, and build a responsive frontend UI featuring a chat interface and secure document upload zone.
Week 3	AI Integration & RAG Pipeline	Implement text chunking (PyPDF), integrate local Ollama (Private Mode) and Cloud APIs (Smart Mode), and build the dynamic hybrid routing logic.
Week 4	Testing, Optimization & Deployment	Perform RAG accuracy testing to eliminate hallucinations, validate offline privacy, fix UI bugs, deploy the application, and prepare final documentation.

Chapter 4

RESULTS AND DISCUSSION

4.1 Expected Outcome

- **Fully Functional Hybrid AI System:** A working prototype that integrates both a local, offline AI engine (via Ollama) and a cloud-based AI API, controlled by a dynamic routing mechanism based on data sensitivity.

- **Guaranteed Data Privacy:** The deployment of a "Private Mode" that allows users to upload, process, and query highly confidential documents entirely offline, ensuring zero data leakage to external servers.

- **Elimination of AI Hallucinations:** A highly accurate Retrieval-Augmented Generation (RAG) pipeline that forces the AI to generate answers strictly from the user's uploaded documents rather than pre-trained guesswork.

- **Intuitive User Interface:** A responsive frontend web application featuring a seamless chat interface, secure document drag-and-drop zones, and real-time status notifications for active AI modes.

- **Integrated Educational Hub:** A built-in blog module that successfully educates users on data privacy best practices and effective prompt engineering for document retrieval.

4.2 Final Outcome

The successful implementation of the Hybrid RAG Knowledge Assistant will result in a fully functional AI system capable of intelligently routing user queries between a secure local LLM for sensitive data and a powerful cloud API for general tasks. By utilizing a localized "Private Mode," the system guarantees complete data privacy, ensuring that confidential documents are processed entirely offline without ever connecting to external servers. Furthermore, the optimized Retrieval-Augmented Generation (RAG) pipeline effectively eliminates AI hallucinations by grounding all generated responses strictly within the context of the user's uploaded documents. Users will interact with the system through a robust,

intuitive web interface that features seamless document uploading, interactive querying, and real-time toast notifications for system status. Finally, the inclusion of an integrated educational hub will empower users by guiding them on prompt engineering, system capabilities, and data privacy best practices, ultimately delivering a secure, highly accurate, and user-friendly AI experience.

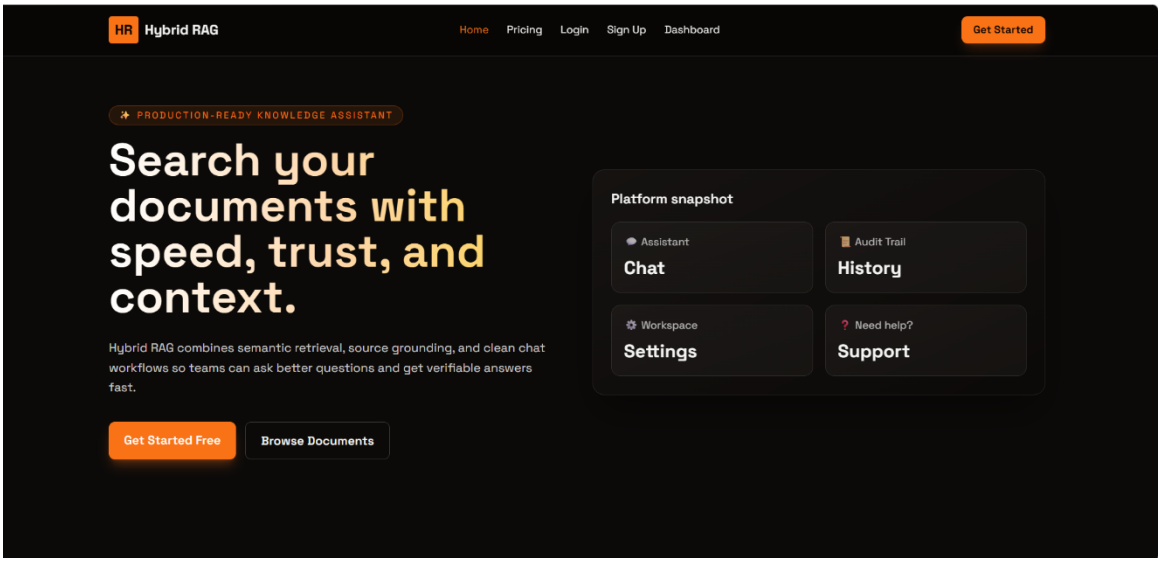

Figure 4.2.1 illustrates the homepage of the Hybrid RAG Knowledge Assistant.

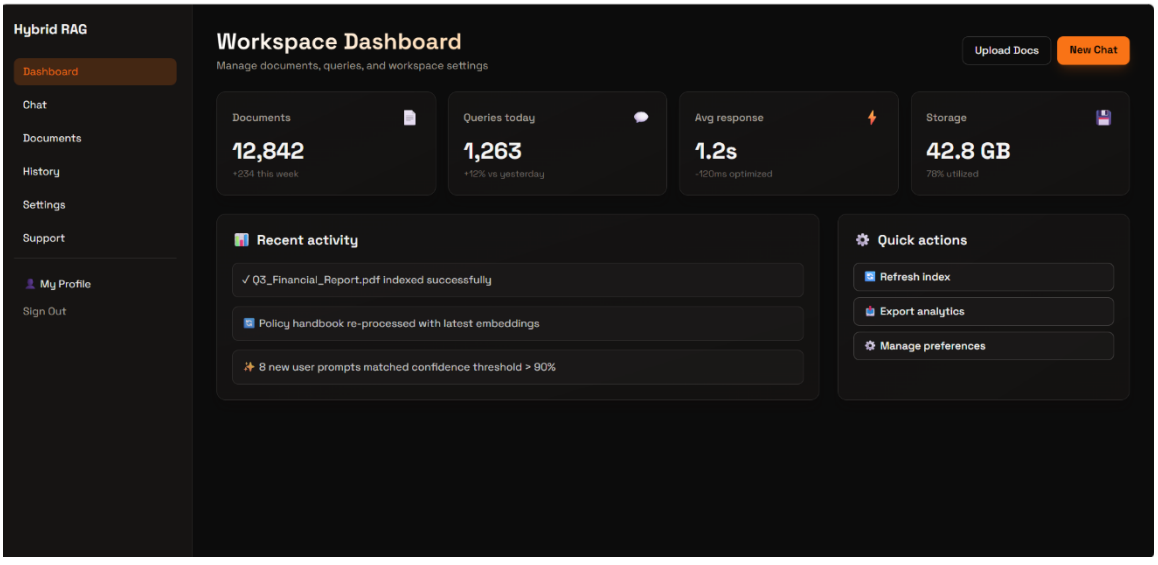

Figure 4.2.2 illustrates the dashboard of the Hybrid RAG Knowledge Assistant.

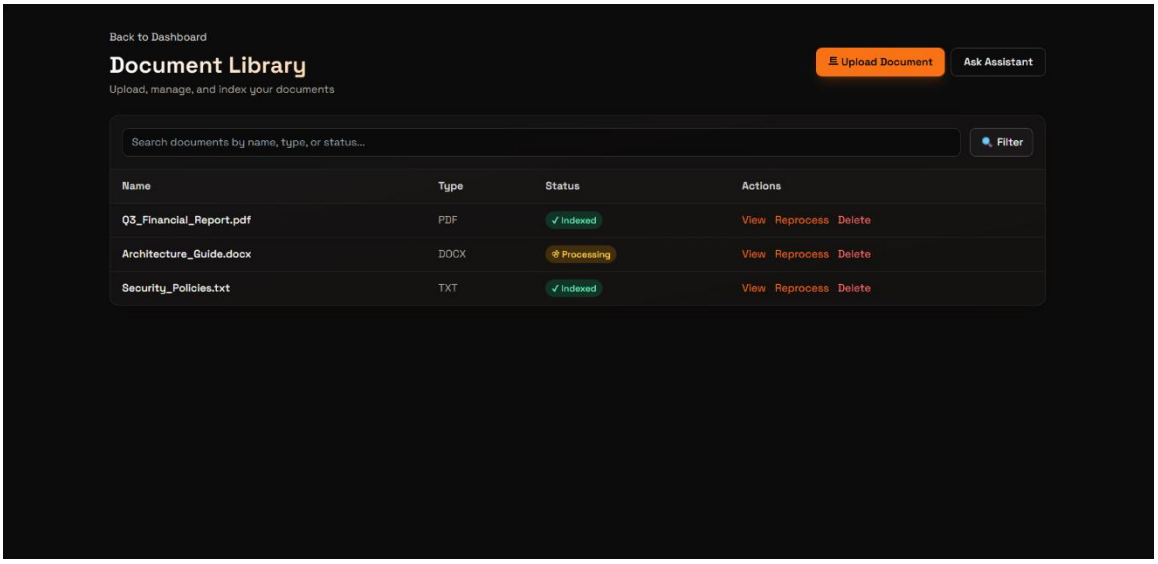

Figure 4.2.3 illustrates the document library of the Hybrid RAG Knowledge Assistant.

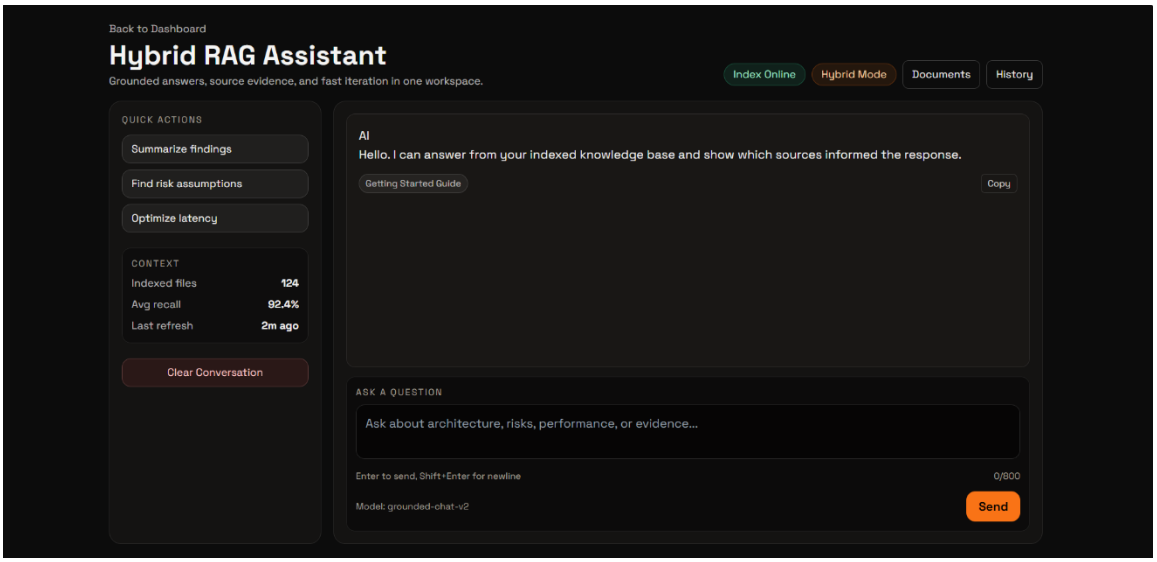

Figure 4.2.4 illustrates the chatpage of the Hybrid RAG Knowledge Assistant.

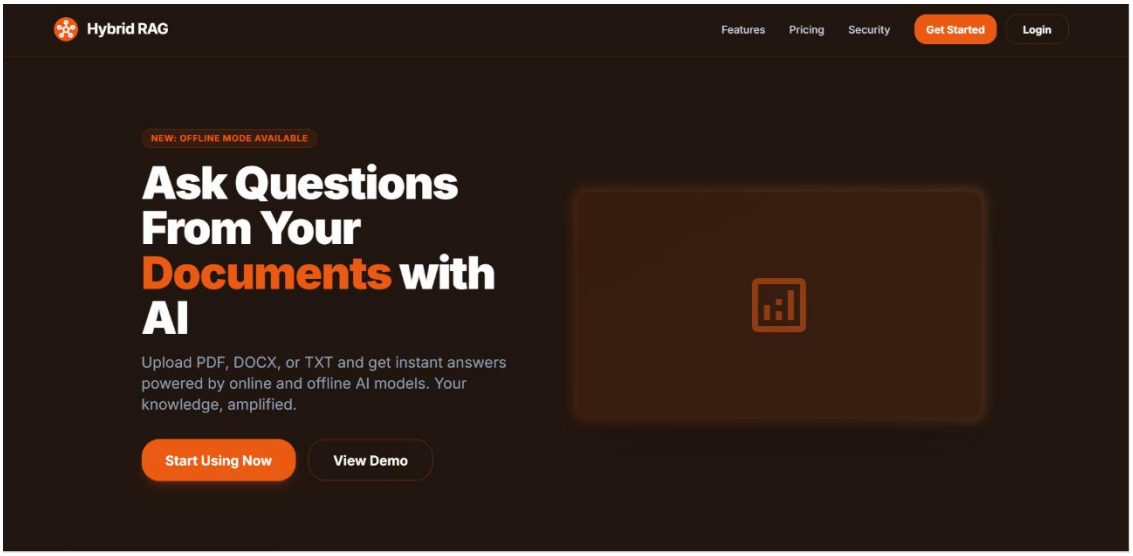

Figure 4.2.5 illustrates the landing page of the Hybrid RAG Knowledge Assistant.

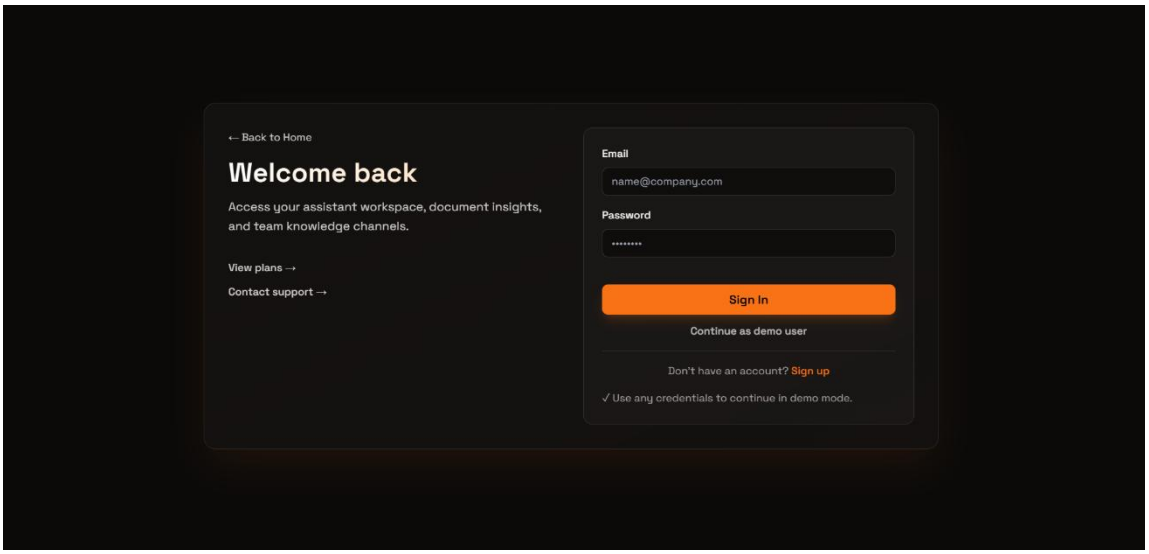

Figure 4.2.6 illustrates the login page of the Hybrid RAG Knowledge Assistant.

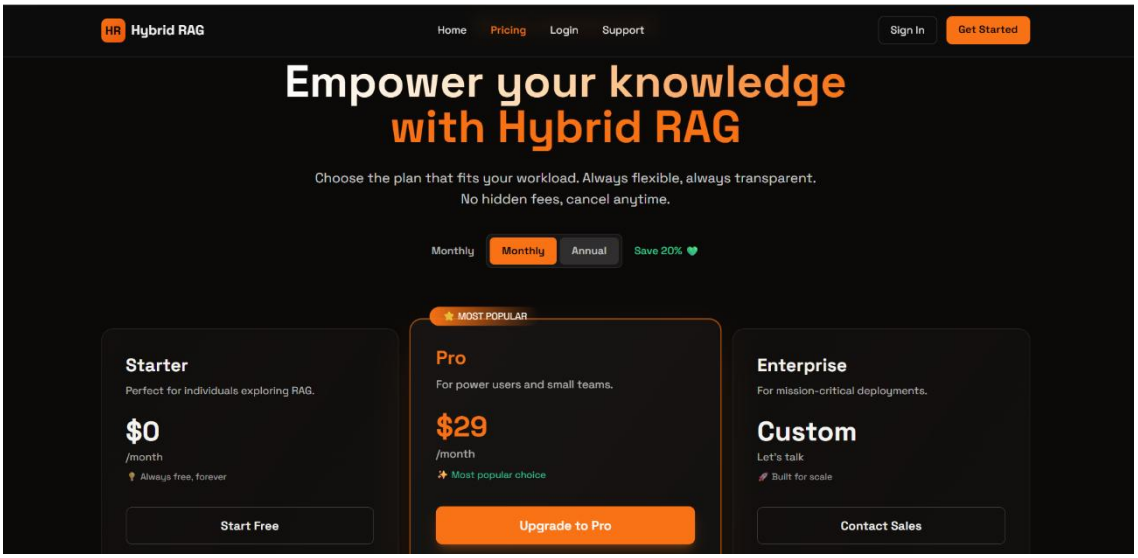

Figure 4.2.7 illustrates the pricing page of the Hybrid RAG Knowledge Assistant.

4.3 Future Scope

- **Multi-Modal RAG:** Support for querying images, charts, and scanned handwritten documents.

- **AI Agent Workflows:** Enabling the AI to perform active tasks like auto-drafting reports and sending emails.

- **Enterprise Integration:** Direct syncing with cloud storage (Google Drive, OneDrive) and team apps (Slack).

- **Live Web Search:** Equipping the Smart Mode with real-time internet access for up-to-date general answers.

- **Mobile App:** Developing a cross-platform mobile application for secure, on-the-go document querying.

Chapter 5

CONCLUSIONS

In conclusion, the Hybrid RAG Knowledge Assistant presents a robust, privacy-centric solution to the critical challenges of data security and AI hallucinations prevalent in modern enterprise environments. By innovating a dynamic, dual-engine architecture, the system successfully bridges the gap between the advanced computational power of cloud-based APIs and the uncompromising security of local, offline Large Language Models. The integration of an optimized Retrieval-Augmented Generation (RAG) pipeline ensures that all generated insights are highly accurate and strictly grounded in user-provided documents, effectively eliminating guesswork. Coupled with an intuitive web interface, real-time status notifications, and an educational resource hub, the project delivers a comprehensive and user-friendly platform. Ultimately, this system empowers users to harness the full potential of cutting-edge AI for document analysis without ever sacrificing the confidentiality of their sensitive data, paving the way for safer and smarter AI adoption in professional workflows. Furthermore, the custom-built hybrid router stands as a testament to the project's technical depth, proving that high-performance AI and strict data governance can seamlessly coexist. As organizations continue to digitize their operations, the scalable nature of this architecture ensures it can easily adapt to larger vector databases and more complex document formats. By drastically reducing the time spent searching through dense PDFs and manuals, the assistant directly contributes to increased productivity and streamlined decision-making. Additionally, the integrated blog empowers users with the necessary skills for prompt engineering, fostering a culture of continuous learning and data awareness. In essence, this project not only demonstrates advanced proficiency in modern AI development but also delivers a practical, market-ready tool tailored for the evolving needs of the digital workplace**.**

Chapter 6

REFERENCES

1. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.** *Advances in Neural Information Processing Systems*, 33, 9459-9474.
2. Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., ... & Zettlemoyer, L. (2019). **BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension.** *arXiv preprint arXiv:1910.13461*.
3. Reimers, N., & Gurevych, I. (2019). **Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.** *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*.
4. Karpukhin, V., Oguz, B., Min, S., Wu, L., Edunov, S., Chen, D., & Yih, W. T. (2020). **Dense Passage Retrieval for Open-Domain Question Answering.** *arXiv preprint arXiv:2004.04906*.
5. Chroma Core. (2024). **Chroma: The AI-native open-source embedding database.** *Official Documentation*. Retrieved from https://docs.trychroma.com/
6. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). **Attention Is All You Need.** *Advances in Neural Information Processing Systems*, 30.
7. LangChain Inc. (2025). **LangChain Official Documentation: Framework for developing applications powered by language models.** Retrieved from https://python.langchain.com/
8. Ollama. (2025). **Ollama: Get up and running with LLaMA 3, Mistral, and other large language models locally.** Retrieved from https://ollama.com/

9. OpenAI. (2025). **OpenAI Platform Documentation: Developer API for GPT-4 and Text Embeddings.** Retrieved from https://platform.openai.com/docs/
10. Ramírez, S. (2025). **FastAPI Documentation: High performance, easy to learn, fast to code API framework.** Retrieved from https://fastapi.tiangolo.com/